Software Requirements Specification (SRS): RAG-Diary

> Version 1.0 of the software requirements specification. Turns the business requirements in the Product Brief into specifications that can be tested.

> Thai version: SRS.md

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of RAG-Diary version 1.0. It is the shared agreement between the development team and the test team on what the system must do and when it is considered correct. Every acceptance criterion is written as Given / When / Then so it can be executed.

1.2 Scope

The system is a web application for private journaling, retrieval by meaning, conversation with one's own entries, and mood summaries expressed as text and images. Scope and exclusions follow section 5 of the Product Brief.

1.3 Definitions and abbreviations

Term	Meaning
RAG	Retrieval-augmented generation: retrieving relevant material to ground an AI answer
Embedding	A numeric vector representing the meaning of text, used to measure semantic closeness
Semantic search	Search by meaning rather than by exact wording
RLS	Row-level security: per-user access control at the row level
MoSCoW	Priority scheme: Must, Should, Could, Won't
AC	Acceptance criteria for a requirement

1.4 References

Document	Relationship
PRODUCT-BRIEF	Source of the business requirements
USE-CASES	Detail of the use cases referenced here
TRACEABILITY-MATRIX	Requirement-to-design-to-test traceability
API-CONTRACT	Interface specification

1.5 Document overview

Section 2 describes the product. Section 3 covers functional requirements, section 4 non-functional requirements, section 5 external interfaces, and section 6 summarises traceability. Section 8 records how each requirement is verified.

2. Overall description

2.1 Product perspective

RAG-Diary is a new, self-built system that works with two external services: Supabase for database and authentication, and an AI provider for embeddings, summaries and images. Users reach it through a browser, and each account's data is fully isolated.

2.2 Capability overview

The system covers account management, writing and managing entries, automatic semantic indexing, semantic search, question-and-answer conversation, mood summaries as text and images, and self-service export and deletion of all data.

2.3 User classes

User	Rights
Guest	Can reach the login and sign-up pages; cannot reach any entry data
Registered user	Can manage all of their own entries and can reach only their own data

2.4 Operating environment

Current browsers on desktop and mobile that support today's standards. The system runs across a server side and a client side.

2.5 Design constraints

- C-01 AI calls cost money per request, so call volume must be limited and usage monitored. The limiter is implemented in the v1.1 code (the rate_limits table) and

awaits the migration before it is active (see ROADMAP and RISK-REGISTER R-02).
- C-02 Some latency depends on external services whose timing cannot be fully controlled.
- C-03 Diary content is sensitive personal data and must comply with the PDPA.

2.6 Assumptions and dependencies

- A-01 Users consent to their entry text being processed by an external AI provider, after being informed.
- A-02 External services (Supabase and the AI provider) are available as agreed.

3. Functional requirements

Priority summary (MoSCoW): Must is required for launch, Should is expected, Could is included when time allows.

ID	Requirement	UC	MoSCoW
FR-01	Register an account	UC-01	Must
FR-02	Log in	UC-01	Must
FR-03	Log out	UC-01	Must
FR-04	Write a new entry with a tag	UC-02	Must
FR-05	Build the semantic index automatically	UC-02	Must
FR-06	View one's own entries	UC-03	Must
FR-07	Edit an entry	UC-03	Should
FR-08	Delete a single entry	UC-03	Must
FR-09	Semantic search	UC-04	Must
FR-10	Chat with one's own entries	UC-04	Should
FR-11	Mood summary as text and an image prompt	UC-05	Should
FR-12	Generate an illustration from the prompt	UC-05	Could
FR-13	Export all data	UC-06	Must
FR-14	Delete all data	UC-06	Must
FR-15	Isolate data per user	Shared by all UCs	Must

FR-01 Register an account

- **MoSCoW:** Must
- **Actor:** Guest

- **Description:** A visitor creates an account with an email and a password.
- **Precondition:** Not signed in.
- **Postcondition:** A new account exists, and the user is either signed in immediately or asked to confirm the email, depending on the provider setting.
- **Acceptance:** Given a visitor enters a password shorter than 6 characters, when they submit, then the system shows an error and creates no account.
- **Acceptance:** Given a visitor enters a valid email and password, when they submit, then the system creates the account and either signs them in or asks them to confirm the email.

FR-02 Log in

- **MoSCoW:** Must
- **Actor:** Guest
- **Description:** A user signs in with an email and password to reach their own data.
- **Precondition:** An account already exists.
- **Postcondition:** The user is signed in and taken to the page they originally intended, if there was one.
- **Acceptance:** Given a signed-out user opens a protected URL directly, when the system checks access, then it redirects to the login page and remembers the intended destination.
- **Acceptance:** Given a user enters a wrong password, when they submit, then the system shows an error and discloses nothing.

FR-03 Log out

- **MoSCoW:** Must
- **Actor:** Registered user
- **Description:** A user signs out and clears cached data on the device.
- **Precondition:** Signed in.
- **Postcondition:** Cached data for the previous account is cleared and the user returns to the guest state.
- **Acceptance:** Given a signed-in user, when they log out, then the system clears all cached data and no longer shows the previous account's entries.

FR-04 Write a new entry with a tag

- **MoSCoW:** Must
- **Actor:** Registered user
- **Description:** A user writes an entry, chooses a tag, and the entry is stored in their account.

- **Precondition:** Signed in.
- **Postcondition:** One new entry exists in the account and the composer is cleared.
- **Acceptance:** Given a signed-in user, when they write text and save, then the system stores the entry and confirms success.
- **Acceptance:** Given saving fails, when the user saves, then the system reports the error instead of staying silent.

FR-05 Build the semantic index automatically

- **MoSCoW:** Must
- **Actor:** System
- **Description:** When a new entry arrives, the system builds its semantic vector and stores it for search.
- **Precondition:** An entry exists without a vector.
- **Postcondition:** That entry can be found by meaning.
- **Acceptance:** Given a new entry is saved, when indexing runs, then the entry appears in semantic search results.

FR-06 View one's own entries

- **MoSCoW:** Must
- **Actor:** Registered user
- **Description:** A user views all of their own entries in time order.
- **Precondition:** Signed in.
- **Postcondition:** Only that user's entries are shown.
- **Acceptance:** Given a user has entries, when they open the list, then they see only their own, newest first.
- **Acceptance:** Given there are no entries, when they open the list, then the system shows a clear empty state rather than a blank screen.

FR-07 Edit an entry

- **MoSCoW:** Should
- **Actor:** Registered user
- **Description:** A user edits the text or tag of an existing entry.
- **Precondition:** The entry belongs to the user.
- **Postcondition:** The entry is updated and the semantic index matches the new text.
- **Acceptance:** Given a user owns the entry, when they edit and save, then the system stores the change and shows the new text.
- **Acceptance:** Given a user tries to edit someone else's entry, when the request is sent, then the system refuses and changes nothing.

FR-08 Delete a single entry

- **MoSCoW:** Must
- **Actor:** Registered user
- **Description:** A user deletes one entry, with confirmation first.
- **Precondition:** The entry belongs to the user.
- **Postcondition:** The entry is deleted and disappears from the list.
- **Acceptance:** Given a user starts a delete and then cancels, when the dialog closes, then the entry is not deleted.
- **Acceptance:** Given a user confirms deletion, when it completes, then the entry is deleted and leaves the list immediately.

FR-09 Semantic search

- **MoSCoW:** Must
- **Actor:** Registered user
- **Description:** A user searches by the meaning of a query rather than by exact words.
- **Precondition:** Signed in, with at least one entry.
- **Postcondition:** Related entries are shown with a similarity score.
- **Acceptance:** Given a query whose words do not appear in the entries but whose meaning is close, when the user searches, then related results are returned.
- **Acceptance:** Given no entry is close enough, when the user searches, then the system says so instead of showing a confusing blank screen.

FR-10 Chat with one's own entries

- **MoSCoW:** Should
- **Actor:** Registered user
- **Description:** A user asks questions about their own entries; the system retrieves relevant entries as context and streams the answer.
- **Precondition:** Signed in.
- **Postcondition:** The user receives an answer grounded in their own entries.
- **Acceptance:** Given a user asks a question, when the system answers, then the response streams and cites context from that user's entries.
- **Acceptance:** Given the AI service fails, when the user asks, then the system shows an understandable error and does not hang.

FR-11 Mood summary as text and an image prompt

- **MoSCoW:** Should
- **Actor:** Registered user

- **Description:** A user picks a date range or tag; the system summarises the overall mood in Thai text and produces an English prompt for image generation.
- **Precondition:** At least one entry falls in the chosen range.
- **Postcondition:** A summary and an image prompt are shown.
- **Acceptance:** Given no entry falls in the range, when the user summarises, then the system says there is nothing to summarise.
- **Acceptance:** Given entries fall in the range, when the user summarises, then the system shows a summary and a clearly separated image prompt.

FR-12 Generate an illustration from the prompt

- **MoSCoW:** Could
- **Actor:** Registered user
- **Description:** A user turns the prompt into an illustration.
- **Precondition:** An image prompt exists.
- **Postcondition:** The generated image is shown.
- **Acceptance:** Given there is no prompt, when the user asks for an image, then the system asks them to summarise first.
- **Acceptance:** Given a prompt exists, when the user asks for an image, then the system shows the image, or an understandable error when the provider does not respond.
- **AC:** Given the image provider has no quota left When the user generates an image Then the page states the reason and offers a mood card drawn on the device instead

FR-13 Export all data

- **MoSCoW:** Must
- **Actor:** Registered user
- **Description:** A user downloads all of their entries as a file.
- **Precondition:** Signed in.
- **Postcondition:** A file containing only that user's data is downloaded.
- **Acceptance:** Given a user exports, when it succeeds, then the file contains only their own entries.

FR-14 Delete all data

- **MoSCoW:** Must
- **Actor:** Registered user
- **Description:** A user permanently deletes all of their entries after an explicit confirmation.
- **Precondition:** Signed in.

- **Postcondition:** All of the user's entries are deleted and the screen updates immediately.
- **Acceptance:** Given a user starts a full delete, when the confirmation states that it is permanent and the user confirms, then every entry is deleted and the screen shows an empty state.
- **Acceptance:** Given deletion succeeded, when the user returns to the list, then no old data remains in the cache.

FR-15 Isolate data per user

- **MoSCoW:** Must
- **Actor:** System
- **Description:** Every data access is limited to the signed-in user's own data.
- **Precondition:** A data request arrives.
- **Postcondition:** The user sees and changes only their own data.
- **Acceptance:** Given user A is signed in, when they try to read or change user B's data, then the system refuses and returns nothing.
- **Acceptance:** Given a request arrives without authentication, when the system processes it, then the request is rejected.

4. Non-functional requirements

ID	Area	Requirement and acceptance
NFR-01	Performance	Semantic search returns within 3.5 seconds at the 95th percentile, with up to 5,000 entries per user
NFR-02	Security	Every data access is authenticated and isolated at the database by RLS, with zero cross-user cases
NFR-03	Privacy (PDPA)	Before text is sent to an external provider the user is informed and consents; the user can export and delete their own data
NFR-04	Usability and accessibility	The product is operable by keyboard, status messages expose an alert role for screen readers, and main colour pairs meet WCAG 2.1 AA contrast
NFR-05	Reliability and error handling	When an external service fails the system shows an understandable error, never a blank or frozen screen, and the user can retry
NFR-06	Scalability	Vector search uses an HNSW index so response time does not rise sharply as data grows
NFR-07	Maintainability	The code is TypeScript, type-checks cleanly, and data access logic comes from a single source rather than being duplicated

ID	Area	Requirement and acceptance
NFR-08	Compatibility and responsive layout	Current browsers are supported and the layout adapts from phone to large screen, in both Thai and English

5. External interfaces

5.1 User interface

The main screens are the home page (write and shortcuts), the memory studio (summary and image), search and chat, history, the user account page, and settings.

5.2 API

The server exposes the core services: semantic search, question-and-answer chat, summaries and image generation. Paths, requests and responses are specified in API-CONTRACT.

5.3 Data interface

An entry holds an ID, the owner, the text, a tag, the creation time and the semantic vector. The structure is specified in DATA-MODEL.

5.4 External service interfaces

Authentication and database come from Supabase; embeddings, summaries and images come from the AI provider. AI calls go through the server, and the secret key is never exposed to the client.

6. Traceability (summary)

Requirements	Use case	Related components
FR-01 to FR-03	UC-01	Authentication system
FR-04 to FR-05	UC-02	Composer and embedding service
FR-06 to FR-08	UC-03	List and history screens
FR-09 to FR-10	UC-04	Search and chat screens
FR-11 to FR-12	UC-05	Studio and image service
FR-13 to FR-14	UC-06	Settings screen
FR-15	Shared by all UCs	RLS policies

The detailed matrix (requirement to screen to API to test case) is in TRACEABILITY-MATRIX.

7. Appendix: MoSCoW meaning

Level	Meaning
Must	Required for launch; without it version 1.0 is not done
Should	Important and expected, but movable under severe constraints
Could	Added value when time allows
Won't	Agreed not to be built in this version

8. Verification

This section states how each requirement is verified, following ISO/IEC/IEEE 29148 and its four standard methods: test, inspection, demonstration and analysis.

8.1 Verifying the functional requirements

ID	Requirement	Method	Level	Pass criteria
FR-01	Register an account	Test	Integration	The account is created, an httpOnly session cookie is set, and a duplicate email is refused
FR-02	Log in	Test	Integration	Correct credentials set the session cookie; wrong credentials return 401
FR-03	Log out	Test	Unit and integration	The server clears the httpOnly cookies and the client clears its cache immediately; requests with no cookie or a bad cookie return 401 (server-side token revocation is out of scope for v1.0)
FR-04	Write a new entry with a tag	Test	Integration and UI	The entry is stored with its tag and appears in the list
FR-05	Build the semantic index automatically	Test	Integration	An embedding is created every time an entry is written
FR-06	View one's own entries	Test	Integration	The list shows only the signed-in user's entries
FR-07	Edit an entry	Test	Integration	The change is stored and reads back
FR-08	Delete a single entry	Test	Integration	The selected entry is deleted and leaves the list
FR-09	Semantic search	Test	Integration	A query whose words are absent from the text still returns related results
FR-10	Chat with one's own entries	Test	Integration	Answers are grounded only in the signed-in user's entries

ID	Requirement	Method	Level	Pass criteria
FR-11	Mood summary as text and an image prompt	Test	Integration	A non-empty summary and prompt are returned
FR-12	Generate an illustration from the prompt	Test	Integration	An image is returned when the provider responds
FR-13	Export all data	Test	Integration	The exported file contains all of the user's entries
FR-14	Delete all data	Test	Integration	The user's data is gone and cannot be read back
FR-15	Isolate data per user	Test	Security test	No case exists where one user can read another's data

8.2 Verifying the non-functional requirements

ID	Requirement	Method	Pass criteria
NFR-01	Performance	Analysis and load test	Semantic search stays within 3.5 seconds at p95
NFR-02	Security	Security test	Zero cross-user access, and every endpoint requires authentication
NFR-03	Privacy (PDPA)	Inspection and demonstration	Consent is requested and recorded, and export and deletion work
NFR-04	Usability and accessibility	Inspection and keyboard test	Every flow works by keyboard, and status messages are readable by a screen reader
NFR-05	Error handling	Failure injection test	When an external service fails the system shows an understandable message and allows a retry
NFR-06	Scalability	Analysis and load test	Response time does not rise sharply as the number of entries grows
NFR-07	Maintainability	Inspection	Type checking passes and no duplicated data access logic remains
NFR-08	Compatibility and responsive layout	Cross-browser test	Correct rendering in current browsers, from phone to large screen

8.3 Acceptance summary

- Every Must requirement passes verification before version 1.0 is considered ready.
- At least 80 percent of Should requirements pass.

- Every verification run is recorded in TEST-PLAN and summarised in UAT.